TripMind

Feature Bibl · All 21 Features Explained

What each feature does · How it impacts the user · How to build it

21 Features	**3 Modules**	**0 Paid APIs**	**100% Offline-Capable**

For Antigravity · Hackathon Submission · March 2026

Module 1 — Smart Itinerary Agent

8 features · builds, reads, fixes your trip

Module 2 — Multilingual + Local Intelligence

6 features · speaks your language · knows local rules

Module 3 — ✨ Disruption Shield

7 features · fixes your trip before you panic

#01

Module: Itinerary

Live Itinerary Engine

Your trip plan is never frozen — it reacts to the real world in real time.

WHAT IT DOES

Subscribes to live data feeds — weather forecasts, venue opening hours, flight status updates. When any real-world condition changes (rain forecast, venue closes early, flight delayed), the itinerary automatically patches the affected events without the user doing anything.

USER IMPACT

Eliminates the moment every traveller knows: arriving at a place to find it's closed, or stepping outside into rain you didn't plan for. The plan is always current. Zero manual update required.

HOW TO BUILD

1. On itinerary load, register background polling for: Open-Meteo (weather, every 3h), Overpass API (venue hours, once/day), flight status (every 30min).
2. On any change, compare with stored ItineraryDay objects.
3. If mismatch → call Ollama LLM with updated context → regenerate affected events.
4. Push updated itinerary to UI via

STACK / APIs →

Open-Meteo (weather, free, no key) · Overpass API (venue data, free) · OpenSky Network (flight tracking, free) · Ollama LLM (local, offline)

#02

Module: Itinerary

Itinerary Auto-Healer

One disruption. Full cascade fix. Flight → hotel → cab → schedule. All automatic.

WHAT IT DOES

When a disruption is detected (flight cancelled, delayed, missed), the system does not just flag the broken event. It identifies every downstream event that is now invalid — hotel check-in time, pre-booked cab, restaurant booking, day activities — and replans the entire affected timeline from scratch.

USER IMPACT

Before TripMind: flight cancelled → 45 minutes of panicking to manually fix 6 things. After TripMind: one notification saying 'We've fixed everything. Here's your new plan.' The emotional cost of disruption drops to near zero.

HOW TO BUILD

1. Disruption event triggers TriggerDisruptionShield use-case.
2. Load all ItineraryDay records for affected dates.
3. Identify all downstream bookings (hotel, cab, activities) with timestamps after the disruption.
4. For each affected booking: calculate new time = disruption_resolution_time + buffer.

to UI.

STACK / APIs →

Internal cascading use-case logic · Prisma DB (SQLite/Postgres) · Ollama LLM (local) · Mock flight data JSON (30 flights, 6 routes)

#03

Module: Itinerary

Calendar Gap Detector

Finds your free time. Fills it with exactly what fits — automatically.

WHAT IT DOES

Reads the user's calendar events (pasted or synced). Detects every gap between 30 and 90 minutes that falls between scheduled events. For each gap, queries nearby places (food, activity, rest) filtered by current GPS location, time of day, and trip purpose (business vs leisure). Pushes 3 ranked suggestions directly into the itinerary timeline.

USER IMPACT

Business travellers stop wasting lunch hours sitting in hotel rooms. Leisure travellers discover local spots they would never have found. The app becomes a proactive assistant, not a passive calendar.

HOW TO BUILD

1. Parse calendarEvents[] array, sort by start time.
2. Iterate pairs: if gap between event[n].end and event[n+1].start is 30–90min → store as FreeGap.
3. For each FreeGap: call Foursquare Places API (or Overpass) with lat/lng + radius 1km + category filter.
4. Pass gap duration + trip purpose to Ollama → returns 3 ranked suggestions.

STACK / APIs →

Calendar API (Google Calendar free tier) or manual paste · Foursquare Places API (1000 calls/day free) · Overpass API (fallback, free) · Ollama LLM

#04

Module: Itinerary

Social Save Ingestor

Your Instagram saves, TikTok bookmarks, Reddit threads → your actual itinerary.

WHAT IT DOES

User pastes text or links from anywhere they save travel content: Instagram captions, TikTok descriptions, Google Maps star lists, Reddit threads. The AI parses all of it, extracts place names and addresses, deduplicates them, geocodes each one, and slots them into the itinerary days based on geographic proximity and time availability.

USER IMPACT

Solves the #1 travel planning pain point: everyone has 300 saved posts and zero organised trip. Your personal taste — the things YOU saved — becomes the actual plan. Not a generic top-10 list generated for a stranger.

HOW TO BUILD

1. User pastes raw text into a textarea.
2. Send to Ollama with prompt: 'Extract all place names, addresses, and cities from this text. Return JSON array: [{name, address, city, category}]'.
3. For each extracted place: call Nominatim geocoding API → get lat/lng.
4. Group places by city + date availability.
5. Score each place against existing ...

slots.

STACK / APIs →

Ollama LLM (local, extracts places from freeform text) · Nominatim / OpenStreetMap (geocoding, free, no key) · Prisma DB to store extracted places

#05

Module: Itinerary

Breathing Room Optimizer

Detects over-packed days. Inserts recovery time before you burn out.

WHAT IT DOES

Analyses each itinerary day and calculates total scheduled activity time. If a day exceeds 5 activities in 8 hours (a configurable threshold), it automatically inserts a 'breathing room' event — a quiet café, a park, a scenic walk — between the most demanding pair of activities. The event is clearly tagged and can be deleted by the user.

USER IMPACT

Travel exhaustion is the biggest reason trips feel disappointing despite going to amazing places. This feature enforces rest without the user having to plan it. By trip day 3, the difference between an over-packed itinerary and a breathing-room-optimized one is enormous.

HOW TO BUILD

1. After itinerary generation, iterate each ItineraryDay.
2. Calculate: sum of event.duration_minutes for activity-type events.
3. If total > 480 mins OR event count > 5: find the pair with largest consecutive gap.
4. Call Overpass API: query for cafes/parks within 500m of that gap's location.
6. Render with distinct 'Breathing room'

STACK / APIs →

Internal calculation logic (no API needed for detection) · Overpass API (find nearby quiet spots, free) · Ollama LLM (generate natural-language description for the break)

#06

Module: Itinerary

Context Engine

Business or leisure. Morning or night. Energetic or recovering. The app adapts.

WHAT IT DOES

Three context inputs feed every recommendation: trip purpose (business vs leisure), time of day (morning/afternoon/evening), and a quick mood tap (energetic / moderate / need rest). These three signals are passed into every Ollama prompt as constraints — so the same city at the same time gives a business traveller and a leisure traveller completely

USER IMPACT

Recommendations feel personal rather than generic. A business traveller at 12:30 PM gets 'quick hawker lunch, 45 minutes, back for your 2 PM meeting.' A leisure traveller gets 'explore the wet market, try the Michelin chicken rice, no rush.' Same location. Completely different experience.

HOW TO BUILD

1. Store tripPurpose (business/leisure) on User model.
2. Derive timeOfDay from current system time (morning/afternoon/evening).
3. Mood tap: 3-button UI component → stores energyLevel (high/medium/low).
4. Compose context string: 'Trip purpose: {purpose}. Time: {timeOfDay}. Energy: {level}.'

STACK / APIs →

No external API — pure LLM prompt engineering · Ollama LLM (local) · localStorage for mood state · User DB record for trip purpose

suggestions, durations, and activity

#07 | Module: Itinerary | NEW ✦

Smart Packing AI

Not a generic packing list. Knows what you're doing each day and packs for that.

WHAT IT DOES

Reads the user's complete itinerary (every event, every day) and the 7-day weather forecast for the destination. Generates a packing list that is specific to what is actually scheduled. Temple visit Day 2 → 'modest clothing that covers shoulders.' Beach Day 3 → 'reef-safe sunscreen, swimwear, flip flops.' Business dinner Day 4 → 'formal outfit, closed shoes.' Rainy Day 5 → 'umbrella,

USER IMPACT

Eliminates the classic travel mistake: arriving at a temple in shorts, or at a beach resort with no sunscreen. Most people pack based on vague destination knowledge. This packs based on your specific schedule — far more accurate and far more useful.

HOW TO BUILD

1. After itinerary is finalised, call Open-Meteo API for 7-day forecast at destination lat/lng.
2. Map each day's weather to the scheduled itinerary events.
3. Build a prompt: 'Given these events per day and this weather forecast, generate a packing list grouped by category (clothing, toiletries, documents, tech, misc). Be specific to the activities.'

checkboxes (localStorage to persist

STACK / APIs →

Open-Meteo API (7-day forecast by lat/lng, free, no key) · Ollama LLM (local, generates event-aware packing list) · localStorage (checklist state)

#08

Module: Itinerary

Visa + Passport + Doc Checklist

Know exactly what you need before you go. Specific to your passport and destination.

WHAT IT DOES

Before the trip, generates a complete pre-travel checklist using the user's passport country and destination country as inputs. Covers: visa requirement and type, passport validity needed, recommended vaccines, travel insurance notes, local emergency numbers, entry rules (biometric scan, arrival card), currency to carry, and any destination-specific advisories.

USER IMPACT

Catches the expensive mistakes before they happen. Arriving at the airport without a visa, or with a passport expiring in 3 months when the destination requires 6 months validity, are both catastrophic. This feature makes pre-trip preparation foolproof.

HOW TO BUILD

1. User inputs passport country + destination country during trip creation.
2. Call Travelbriefing.org API (free) with country pair → returns visa info, health, safety.
3. Call REST Countries API (free, no key) → returns capital, currency, emergency numbers.
4. Pass all raw data to Ollama with prompt: 'Convert this into a clear,

6. Mark items complete with

STACK / APIs →

Travelbriefing.org API (visa + health info, free) · REST Countries API (country data, free, no key) · Ollama LLM (formats into readable checklist)

#09

Module: Intelligence

Multilingual UI

The whole product speaks your language — not just the buttons. The AI too.

WHAT IT DOES

The entire product renders in the user's chosen language: English, Hindi, Spanish, French, Japanese. This is not just UI label translation. Every AI-generated response — itinerary descriptions, cultural nudges, disruption alerts, packing lists — is produced in the user's language by instructing the LLM to respond in that language. Language can be changed at any point and the

USER IMPACT

For non-English speakers (the majority of the world), this changes the product from 'usable with effort' to 'feels built for me.' A Hindi-speaking traveller gets every itinerary description, every alert, every suggestion in Hindi without thinking about it.

HOW TO BUILD

1. Install i18next + react-i18next.
2. Create translation JSON files: en.json, hi.json, es.json, fr.json, ja.json — covering all static UI strings.
3. Detect language from browser headers or user preference stored in DB.
4. For all dynamic LLM content: append 'Respond entirely in {language}. Do not use English.' to every Ollama prompt.
6. On switch: i18next.changeLanguage()

STACK / APIs →

i18next + react-i18next (npm, free) · Ollama LLM (prompted in target language) · 5 translation JSON files (static UI strings) · User.preferredLang DB field

#10 | Module: Intelligence | NEW ✦

Multilingual Voice + Live Translate

Speak. Point. Understand. In any language. Fully offline.

WHAT IT DOES

Three capabilities in one: (1) Voice input — speak your request in any language, it transcribes and responds via voice. (2) Camera translate — point your phone camera at a foreign menu, sign, or notice — it translates live on screen. (3) Conversation interpreter — you speak, the app speaks back in the local language for the person in front of you. A real-time translator in your pocket.

USER IMPACT

Ordering food in Sri Lanka without knowing Sinhala. Understanding a safety notice in Japanese. Having a conversation with a local cab driver in their language. These are the friction-filled moments every traveller faces. This feature eliminates all of them, entirely offline.

HOW TO BUILD

1. Voice input: use Web Speech API (browser built-in, free) or Whisper.cpp (local Ollama model) for STT.
2. Translation: run LibreTranslate in Docker on localhost:5000 (docker run -p 5000:5000 libretranslate/libretranslate). No internet needed.
3. Camera translate: use device camera API → capture frame → send image to Ollama vision model (llava) → returns

5. All three modes work offline after

STACK / APIs →

Web Speech API (browser built-in, STT + TTS) · LibreTranslate (self-hosted Docker, offline, 30+ languages) · Ollama llava model (vision + translation, local)

#11

Module: Intelligence

Expense AI

Paste a receipt. It categorises itself. Your expense report writes itself.

WHAT IT DOES

User pastes raw receipt text (or in a future version, photographs it). The AI reads the unstructured text and extracts: total amount, currency, merchant name, category (food / transport / accommodation / activity / other), and date. All expenses are stored against the trip. By the end of the trip, a complete expense report is ready to submit — with category breakdown

USER IMPACT

63% of business travellers say manual expense reporting is their most hated travel task (GBTA 2025). This turns a 45-minute post-trip chore into something that happens automatically as you go. You land home with a finished report.

HOW TO BUILD

1. Textarea input for receipt text (or future: OCR from image via Tesseract.js).
2. Ollama prompt: 'Extract from this receipt: amount (number), currency (3-letter code), category (food/transport/accommodation/activity/other), description (merchant + item), date (YYYY-MM-DD). Return only valid JSON.'

STACK / APIs →

Ollama LLM (local, extracts structured data from receipt text) · ExchangeRate-API (1500 free calls/month, currency conversion) · Tesseract.js (OCR, future) · Recharts (chart)

req/month) for currency conversion to

#12 | Module: Intelligence | NEW ✦

Memory Lane (Past Trips + Google Photos)

Your travel history makes every future trip smarter. Learns your taste over time.

WHAT IT DOES

Connects to Google Photos (or accepts manual photo uploads) and scans past trip photos using GPS metadata embedded in each photo. Builds a 'travel preference profile' from where the user actually went and what they photographed most. This profile — 'loves rooftop bars, avoids museums, prefers street food, morning person' — is passed into every future LLM prompt

USER IMPACT

'You loved rooftop bars in Goa — here's one in Singapore' is infinitely more powerful than 'popular bars in Singapore.' The app gets smarter with every trip. After 3 trips, it knows you better than any travel agent.

HOW TO BUILD

1. Google Photos API (free, OAuth) or accept photo folder upload.
2. Parse EXIF data from each photo: GPSLatitude, GPSLongitude, DateTimeOriginal.
3. Reverse geocode each cluster of photos → identify places visited.
4. Send location list to Ollama: 'Based on these places visited on past trips, identify travel preferences and patterns.

User.travelProfile JSON field.

STACK / APIs →

Google Photos API (free, OAuth required) · EXIF data parsing (exifr npm package) · Nominatim reverse geocoding (free, no key) · Ollama LLM (profile generation)

#13

Module: Intelligence

Local Law Nudges + Rights-Checker

Know what's acceptable before you do something accidentally offensive or illegal.

WHAT IT DOES

Two modes: (1) Passive nudges — small contextual alerts appear directly on relevant itinerary event cards: 'Photography restricted inside this monument.' 'Dress code: cover shoulders and knees.' 'Alcohol not served here on Fridays.' (2) Active chatbot — user types 'Is it legal to take photos of police in Singapore?' and gets a plain-language answer with cultural

USER IMPACT

The difference between an embarrassing moment (being turned away from a temple in shorts) and a serious one (unknowingly breaking a local law). For first-time visitors to Southeast Asia, the Middle East, or Japan, this feature alone prevents multiple uncomfortable incidents per trip.

HOW TO BUILD

1. Passive nudges: build a JSON dataset of 200+ venue-type + country pairs with associated rules (can be LLM-generated once and cached).
2. When an event is added to the itinerary, match its category + destination country against the dataset.
3. If match found: attach nudge text to event object → render as small notice on event card.

STACK / APIs →

Static JSON dataset (venue rules by country, one-time LLM generation) · Ollama LLM (chatbot answers) · REST Countries API (country context)

Give a practical plain-language answer

#14 | Module: Intelligence | NEW ✦

Heatmap

See crowds, prices, and safety at a glance. Decide where to go before you go.

WHAT IT DOES

An interactive map with three toggleable overlay layers: (1) Crowd density heatmap — shows how busy each area of the city is, segmented by time of day (morning/afternoon/evening/night), sourced from OpenStreetMap and Overpass foot-traffic proxy data. (2) Price heatmap — colour-codes neighbourhoods from cheap green to expensive red based on average venue

USER IMPACT

Answers the questions every traveller asks before choosing where to eat or stay: 'Is this neighbourhood too touristy at 12PM?' 'Are restaurants near my hotel expensive?' 'Is it safe to walk here at 9PM?' All answered visually in 5 seconds rather than 30 minutes of Google research.

HOW TO BUILD

1. Base map: Leaflet.js + OpenStreetMap tiles (both free, no key).
2. Crowd data: query Overpass API for POI density per grid cell (higher POI density = proxy for crowding). Segment by time category.
3. Price data: query Foursquare Places API for average price tier per neighbourhood grid.
4. Safety data: use public crime/incident

STACK / APIs →

Leaflet.js (open source map library) · OpenStreetMap tiles (free, no key) · Overpass API (POI density, free) · Foursquare Places (price data, 1000 free/day) · Leaflet.heat plugin

Leaflet.heat plugin (open source).

#15

Module: Disruption

Proactive Delay Predictor

Warned 3 hours before disruption. You rebook before the crowd even notices.

WHAT IT DOES

Monitors three data streams for every upcoming flight in the user's trip: (1) weather at origin and destination airports, (2) historical delay rate for that specific flight number and day-of-week, (3) live flight tracking via OpenSky Network. When all three signals converge on likely disruption, an alert is sent up to 3 hours before the scheduled departure — giving the user a significant

USER IMPACT

Reactive disruption management (you find out when it happens) vs predictive (you find out before it's certain). The difference in outcomes is enormous. You rebook while seats are still available. You are not competing with 200 other passengers for 12 remaining seats.

HOW TO BUILD

1. For each FlightBooking: schedule background job 24h before departure.
2. Open-Meteo API: fetch weather for origin + destination airports for departure day.
3. OpenSky Network API: query current aircraft status + recent delay history for flight number.
4. Build a delay risk score: weather_risk (0-1) + historical_delay_rate (0-1) →

6. Store alert in DB to avoid duplicate

STACK / APIs →

Open-Meteo API (weather, free, no key) · OpenSky Network API (live flight data, free, no key) · node-cron (background job scheduling) · WebSocket or polling for real-time alert push

#16

Module: Disruption

Auto Flight Alternative Holder

Disruption detected → 3 alternatives held → best one pre-selected → you just confirm.

WHAT IT DOES

The moment a disruption is confirmed, the system queries the flight dataset for alternatives on the same route within ±6 hours of the original departure. Returns exactly 3 options, each scored by a weighted formula: 40% weight on earliest arrival time (most important — time is precious during disruption), 30% on price difference from original, 30% on whether the seat type matches user

USER IMPACT

Compresses the most stressful 45 minutes of any trip into a 10-second decision. You don't research, you don't search, you don't compare. Three pre-evaluated options appear. Tap the one you want. Done.

HOW TO BUILD

1. On disruption trigger: call MockFlightService.findAlternatives(origin, dest, date, maxResults: 3).
2. MockFlightService queries static JSON of 30 flights filtered by route + time window.
3. For each alternative: compute score = (0.4 × arrivalEarliness) + (0.3 × (1 - priceDelta/originalPrice)) + (0.3 × seatMatch).

card gets amber border + 'Best match'

STACK / APIs →

MockFlightService (static JSON, 30 flights, 6 routes — no API needed for demo) · AviationStack API (real data, 100 free calls/month — use for live demo if available)

#17

Module: Disruption

Hotel + Cab Auto Time-Shift

New flight time → hotel and cab update themselves. Zero phone calls.

WHAT IT DOES

After a new flight is selected, the system automatically recalculates all time-sensitive bookings. Hotel check-in: new arrival time + 2 hours buffer = new check-in time. If the hotel cannot hold until the new time, an alternative nearby hotel with available rooms is flagged. Pre-booked cab: new pickup time = new arrival time at airport + standard baggage claim duration (45 min). All

USER IMPACT

The scenario TripMind eliminates: landing at 8 PM when your hotel check-in closed at 4 PM, finding your room released, making 6 phone calls from an airport to sort accommodation. This feature makes that scenario impossible.

HOW TO BUILD

1. After new flight confirmed: retrieve associated HotelBooking and CabBooking for the trip.
2. Calculate: newHotelCheckIn = selectedFlight.arrivalTime + 120 minutes.
3. Update HotelBooking.checkIn in DB.
4. Calculate: newCabPickup = selectedFlight.arrivalTime + 45 minutes (baggage claim buffer).

STACK / APIs →

Internal DB update logic (Prisma) · Overpass API (fallback hotel search if needed) · No external booking API needed for demo — mock data

hotel.latestCheckIn (from venue data):

#18

Module: Disruption

AI Agent Full Auto-Rebook

Flight cancelled → agent fixes everything → sends you a new complete itinerary. Autonomous.

WHAT IT DOES

The complete autonomous loop. When a disruption is confirmed: (1) agent finds best alternative flight from dataset, (2) books it (mock confirmation), (3) shifts hotel check-in, (4) reschedules cab, (5) rebuilds the entire affected day's itinerary around the new timeline, (6) sends the user a new complete day plan. The user receives one notification when everything is done: 'We've sorted

USER IMPACT

This is the product's core promise in its most complete form. The contrast with current reality is stark: today, a flight cancellation requires 45+ minutes of the user's time and significant stress. TripMind reduces that to one tap to confirm. The agent does everything else.

HOW TO BUILD

1. Chain all disruption use-cases in sequence: TriggerDisruptionShield → selectBestAlternative → updateHotelBooking → updateCabBooking → rebuildItineraryDay.
2. Run as async pipeline with status events broadcast to UI via WebSocket.
3. UI shows a 'Resolving disruption...' animation with step-by-step progress:

DisruptionResolution object to UI.

STACK / APIs →

Internal async pipeline (Node.js async/await chain) · WebSocket (real-time progress updates to UI) · Ollama LLM (rebuild affected itinerary day) · Prisma DB

#19

Module: Disruption

QR Confirm / Cancel

One QR. Two buttons. Confirm & Pay or Cancel. That's it. Works offline.

WHAT IT DOES

After all disruption resolution is complete, the user receives a boarding-pass-styled QR card. The QR encodes a confirmation URL with a unique short token: tripmind.app/confirm/{token}?action=pay&amount=4200&currency=INR. Two buttons below the QR: 'Confirm & Pay ₹4,200' (amber primary button) and 'Cancel booking' (red ghost button). The

USER IMPACT

This is the moment that is remembered. Every other solution — airlines, travel apps, TMCs — ends their disruption flow with a form, a phone call, or a multi-step checkout. TripMind ends it with a QR card and two buttons. The simplicity is the point. Judges and users both feel the contrast immediately.

HOW TO BUILD

1. Generate UUID confirmation token: crypto.randomUUID().
2. Build confirmation URL: https://tripmind.app/confirm/{token}?action=pay&amount={price}&currency={curr}&flight={flightId}.
3. npm install qrcode → call QRCode.toDataURL(confirmationURL) → returns base64 PNG.
4. Render QR card component: white

STACK / APIs → qrcode npm package (local, zero internet, zero cost) · crypto.randomUUID() (built-in Node.js) · Internal confirm/cancel API endpoints

no failure risk on demo day.

5. 'Confirm' button: POST

#20 | Module: Disruption | NEW ✦

Instant Cancellation Guardian

Your Sri Lanka moment — return flight cancelled mid-trip — handled before you even open the app.

WHAT IT DOES

Monitors ALL bookings throughout the entire trip duration — not just outbound flights, but return flights and hotel stays too. The moment a cancellation notification arrives (detected via email parsing or webhook simulation), the system immediately holds an alternative seat on a comparable flight before the user sees the message. It also checks that the user's linked payment method

USER IMPACT

This feature was built from a real experience: being in Sri Lanka, enjoying the trip, and suddenly getting a message saying the return ticket was cancelled. The panic of that moment — being in a foreign country, needing to rebook, hoping you have enough money — is exactly what this feature eliminates. The alternative is held before you panic. The balance is checked before you commit.

HOW TO BUILD

1. Schedule monitoring job for ALL FlightBooking records with departure in future.
2. Two detection methods: (a) Poll AviationStack API for booking status changes every 30 min, (b) Simulated webhook for demo: UI button 'Simulate cancellation'.
3. On cancellation detected: immediately call MockFlightService.findAlternatives()

STACK / APIs →

AviationStack API (booking status, 100 free calls/month) · MockFlightService (demo fallback, always works offline) · node-cron (polling job) · User.paymentBalance mock field

held SQ422 at ₹8,200 — confirm or

confirming hold.

#21

Module: Disruption

Full Itinerary Resend After Disruption

You never see a broken schedule. Every disruption ends with a complete, coherent plan.

WHAT IT DOES

After any disruption is fully resolved (new flight confirmed, hotel shifted, cab rescheduled), the affected itinerary days are completely rebuilt from scratch — not just patched. The Ollama LLM is called with the new time constraints and rebuilds a coherent sequence of events that fits the new timeline. If the user now lands at 6:30 PM instead of 2:00 PM, the afternoon museum visit is removed,

USER IMPACT

The feeling of being in control even when things go wrong. A patched itinerary (where some events now have wrong times) is almost as stressful as no itinerary. A fully rebuilt itinerary that makes sense again is genuinely reassuring. This is the difference between a band-aid and a fix.

HOW TO BUILD

1. After DisruptionResolution is confirmed: delete all ItineraryDay.events[] for affected dates.
2. Build new prompt context: original trip preferences + new flight arrival time + remaining trip duration.
3. Call Ollama: 'Rebuild day plan for {date}. User arrives at {location} at {newArrivalTime}. Original preferences: {prefs}. Return full day schedule in

5. Push updated itinerary to UI with

STACK / APIs →

Ollama LLM (local, rebuilds full day from new constraints) · Prisma DB (save rebuilt day, preserve previous version) · WebSocket (push updated itinerary to UI in real time)

21 Features. 3 Modules. 0 Paid APIs.

Everything runs offline. The demo never fails.

Module 1

Smart Itinerary Agent

8 features

Module 2

Multilingual + Local Intelligence

6 features

Module 3

✨ Disruption Shield

7 features

Your story	Sri Lanka return flight cancellation → Feature #20
Demo moment	Disruption Shield QR card — works 100% offline
Stack	Ollama + Open-Meteo + OSM + LibreTranslate + qrcode
Cost	₹0 / $0 — zero paid APIs required